

```

import boto3
import cv2
import os
import uuid
import urllib.parse
from datetime import datetime

# --- CONFIGURATION (Ensure these match your actual AWS names) ---
OUTPUT_BUCKET = "surveillance"
MODEL_ARN = "arn:aws:rekognition:"
SNS_TOPIC_ARN = "arn:aws:sns:ap-south-1:Emerge"
TABLE_NAME = "Surveillance"

s3 = boto3.client('s3')
rekognition = boto3.client('rekognition')
sns = boto3.client('sns')
dynamodb = boto3.resource('dynamodb').Table(TABLE_NAME)

def lambda_handler(event, context):
    try:
        # 1. Get video from INPUT bucket
        input_bucket = event['Records'][0]['s3']['bucket']['name']
        key =
        urllib.parse.unquote_plus(event['Records'][0]['s3']['object']['key'])
        download_path = f"/tmp/video.mp4"

        print(f"Processing {key}...")
        s3.download_file(input_bucket, key, download_path)

        cap = cv2.VideoCapture(download_path)

        # Check every 10 frames (approx 0.3 seconds) for accuracy
        frame_interval = 10
        frame_count = 0

        # VARIABLES TO TRACK THE "BEST SHOT"
        max_confidence = 0
        best_frame_bytes = None
        best_labels = []

        while cap.isOpened():
            ret, frame = cap.read()
            if not ret: break

            if frame_count % frame_interval == 0:
                _, buf = cv2.imencode('.jpg', frame)
                img_bytes = buf.tobytes()

                try:
                    # We ask AI for anything above 50% so we can track
                    rising confidence
                    # But we will ONLY alert if it hits 75% later
                    resp = rekognition.detect_custom_labels(
                        ProjectVersionArn=MODEL_ARN,
                        Image={'Bytes': img_bytes},
                        MinConfidence=50
                    )

```

```

        if len(resp['CustomLabels']) > 0:
            current_confidence =
resp['CustomLabels'][0]['Confidence']
            current_labels = [l['Name'] for l in
resp['CustomLabels']]

            # Logic: Is this the best shot so far?
            if current_confidence > max_confidence:
                max_confidence = current_confidence
                best_frame_bytes = img_bytes
                best_labels = current_labels
                print(f"New Best Shot: {current_confidence}%
(Frame {frame_count})")

            # Optimization: If we find a 99% match, we can
stop early
            if max_confidence > 99:
                break

        except Exception as e:
            print(f"AI Error: {e}")

        frame_count += 1

    cap.release()

    # --- THE GATEKEEPER: 75% RULE ---
    print(f"Final Maximum Confidence: {max_confidence}%")

    if max_confidence >= 75: # <--- UPDATED TO 75%
        print("Confidence is above 75%. Sending Alert.")

        # 1. Save the BEST frame
        snapshot_key = f"{uuid.uuid4()}.jpg"
        s3.put_object(
            Bucket=OUTPUT_BUCKET,
            Key=snapshot_key,
            Body=best_frame_bytes,
            ContentType='image/jpeg'
        )

        # 2. Generate Email Link
        snapshot_link = s3.generate_presigned_url(
            'get_object',
            Params={'Bucket': OUTPUT_BUCKET, 'Key': snapshot_key},
            ExpiresIn=604800
        )

        # 3. Save to DB
        dynamodb.put_item(Item={
            'incident_id': str(uuid.uuid4()),
            'timestamp': datetime.now().strftime("%Y-%m-%d
%H:%M:%S"),
            'labels': best_labels,
            'video_file': key,
            'snapshot_key': snapshot_key
        })

```

```

# 4. Send Email
email_body = (
    f" Confidence: {int(max_confidence)}%\n\n"
    f" CLICK TO VIEW BEST EVIDENCE:\n{snapshot_link}"
)

sns.publish(
    TopicArn=SNS_TOPIC_ARN,
    Message=email_body,
    Subject=f" {best_labels[0]} Detected!"
)
else:
    print(f"Alert Skipped. Confidence {max_confidence}% is below
75% threshold.")

    return "Done"

except Exception as e:
    print(f"Critical Error: {e}")
    return "Error"

```